Trade-off Analysis between Accuracy and Computational Efficiency in Sarcasm Detection Using DistilBERT and RoBERTa

Falah Razan Hibrizi
School of Computing
Telkom University
Bandung, Indonesia
email address

Supervisor Name
School of Computing
Telkom University
Bandung, Indonesia
email address

***Abstract*—Sarcasm often expresses the opposite of its literal meaning, which causes sentiment analysis systems to misread sarcastic sentences. RoBERTa is known to be accurate but computationally expensive, whereas DistilBERT offers a smaller model. This study examines the trade-off between performance and computational efficiency of both models on 1,000,864 comment and parent comment pairs from the Self-Annotated Reddit Corpus (SARC). The comparison covers three input configurations, namely comment only, early fusion, and late fusion, as well as partial layer freezing. The final configurations were selected on validation data and locked before any test score was computed. Each configuration was run with three seeds and evaluated with the paired t-test, the Wilcoxon signed-rank test, and the Friedman test with Holm correction. The highest macro-F1 of 79.62% was obtained by RoBERTa with early fusion, and RoBERTa outperformed DistilBERT by 1.65–2.07 points across the three inputs. Nevertheless, DistilBERT still reached 97–98% of the RoBERTa macro-F1 while requiring only about half of the training time, latency, and GPU memory. Adding the parent comment raised macro-F1 by 0.69–1.10 points, with the largest gain on ambiguous comments. For RoBERTa, early fusion was more accurate and far cheaper than late fusion. Freezing 6 of the 12 RoBERTa layers reduced macro-F1 by only 0.24 points and cut training time by about 45%, although inference latency did not decrease. These results indicate that RoBERTa is preferable when accuracy is the priority, whereas DistilBERT is more suitable under latency and memory constraints.**

***Index Terms*—computational efficiency, DistilBERT, layer freezing, parent comment, RoBERTa, sarcasm detection**

I. Introduction

In sarcasm, the speaker says something different from, or even opposite to, what is actually meant. A compliment, for example, can be used to mock. Such sentences are difficult for sentiment analysis systems because words that look positive carry a negative intent [1]. As a result, public opinion monitoring and content moderation systems may produce wrong outputs, including when they are used to detect cyberbullying on social media [2], [3].

The difficulty increases in written text, since there is no tone of voice or facial expression to serve as a cue [4]. On forums such as Reddit, the sarcastic nature of a reply often becomes clear only after the reader knows the comment being replied to, called the parent comment. The Self-Annotated Reddit Corpus (SARC) [5] provides a very large number of parent comment and reply pairs. Several studies on Reddit conversation data have reported that feeding the parent comment into a transformer model can improve sarcasm detection performance [6], [7].

Transformer-based models are currently widely used for this task. RoBERTa [8] is a refinement of BERT with an improved pretraining procedure. It generally achieves high accuracy, but its memory and computation requirements are also large. DistilBERT [9] is obtained through knowledge distillation, so it has half the layers of BERT and faster inference. Several studies have compared BERT variants for sarcasm detection on different types of text [4], [10], and their results show that performance depends strongly on the choice of model and training settings.

A direct comparison of RoBERTa and DistilBERT, including on SARC, was carried out by Dubey et al., who reported that DistilBERT trained about 1.74 times faster with competitive performance [7], [11]. However, both models were combined with BART context summaries and metadata embeddings, which makes it difficult to isolate the contribution of the models themselves. The experiments also used only a subset of 7,370 samples [7], and the preprocessing removed capitalization and punctuation [11], even though both can signal sarcasm. In [11], the performance difference between the two models was not even significant under the paired t-test ($p = 0.064$). Zhang et al. [12] used SARC at a larger scale but did not separate test data from validation data. Meanwhile, the way the parent comment is fed into the model and layer freezing, which can serve as a cost-saving option besides distillation, have rarely been discussed together with their computational cost in the context of sarcasm detection.

Based on these observations, this study compares the performance and computational efficiency of RoBERTa and DistilBERT on approximately one million SARC comment pairs without additional components. The trade-off between the two is then analyzed to determine the conditions best suited to each model. This aim is broken down into the following three

research questions.

1) Does the number of frozen layers affect sarcasm classification performance?
2) Does using the parent comment as input affect sarcasm classification performance?
3) Which fusion strategy yields the best sarcasm classification performance?

The three questions are answered for both models while taking into account the computational cost in terms of training time, inference latency, GPU memory, and number of parameters. The contributions of this study are (i) a comparison of both models with light preprocessing, configurations locked before the test data were opened, three seeds, and statistical tests with Holm correction; (ii) measurements of computational cost alongside performance for three input configurations and several levels of layer freezing; and (iii) an analysis of the types of comments that benefit most from the parent comment.

II. Methodology

A. Dataset and Preprocessing

This study uses the balanced version of SARC [5] released on Kaggle, the same version used in [11] and [12]. It contains 1,010,826 Reddit comments together with the parent comments they reply to. The labels do not come from annotators but from the "/s" marker added by the comment authors themselves, and this marker was removed from the text when the corpus was built [5].

Preprocessing was deliberately kept simple. In previous studies, text was lowercased, punctuation or special characters were removed, and lemmatization was applied [11], [12]. These steps were not applied here, because capitalization, punctuation, and emoticons such as ";)" may indicate sarcasm. Cleaning consisted only of decoding HTML entities, replacing Markdown links with their text, removing URLs and HTML tags, and normalizing whitespace. As a result, only 2,942 comments (0.29%) were changed.

The number of rows removed at each step is shown in Table I. Duplicates and conflicting labels were detected using a normalized key (lowercase and without punctuation), while the text fed to the model remained the original text. In the last step, residual artifacts of the "/s" marker removal were discarded. These artifacts are comments that contain only a sentence fragment, such as "You forgot the". They were matched with a pattern of at most eight words. Of the 8,599 matching rows, 8,580 were labeled sarcastic, so all of them were removed to prevent the model from learning this shortcut. After filtering, 1,000,864 pairs remained, with a sarcastic proportion of 49.59%.

The training, validation, and test split was performed with seed 42. The data were split by comment group rather than by row, so comments with the same key always fall into the same split. This is necessary because short replies such as "Source?" and "Yes." appear hundreds of times in the data. The resulting split is close to an 80/10/10 ratio, and no comment or pair appears in more than one split.

TABLE I
Row Filtering and Data Split

Filtering step	Removed	Remaining
Raw data	–	1,010,826
Empty comment	55	1,010,771
Empty after cleaning	9	1,010,762
Conflicting labels	142	1,010,620
Duplicate pairs	1,157	1,009,463
"/s" fragments	8,599	1,000,864
Split	**Pairs**	**Sarcastic (%)**
Training	799,459	49.66
Validation	100,276	49.38
Test	101,129	49.27

B. Models and Input Configurations

RoBERTa-base [8] consists of 12 Transformer layers with a hidden size of 768 and a total of 124.6 million parameters. DistilBERT [9] is a distilled version of BERT with the same hidden size but only six layers and 65.8 million parameters. The cased version of DistilBERT was chosen so that both models can distinguish capitalization. To examine the effect of the parent comment and the fusion strategy, three input configurations were compared as follows.

Comment only. The model receives only the reply text. This configuration serves as the baseline.

Early fusion. The parent and comment are combined into a single input using the tokenizer's sentence-pair format, with the parent placed first. This approach is similar to the concatenation of context and response in [6]. With this arrangement, every comment token can attend to all parent tokens from the first layer onward. The input length is limited to 256 tokens. When the limit is exceeded, the end of the parent is truncated, which happens for 0.85% of the training pairs for RoBERTa and 0.99% for DistilBERT. Comments longer than 256 tokens are also truncated, but there are only 52–54 such training comments (less than 0.01%).

Late fusion. The parent and comment are encoded separately with the same encoder. The final hidden states at the first token of the comment (h_x) and of the parent (h_y) are then combined into

$$u = [\, h_x;\ h_y;\ |h_x - h_y|;\ h_x \odot h_y \,], \tag{1}$$

where $[\cdot\,;\cdot]$ denotes vector concatenation and $\odot$ denotes element-wise multiplication. The vector u in (1) is passed to the classification head, which follows the default head of each model except that its input size is four times larger. The truncation rules are the same as in early fusion. Consequently, the encoder must be run twice for every example, and comment tokens cannot attend directly to parent tokens.

C. Layer Freezing

In the layer freezing experiments, the embedding layer and a number of the lowest layers were frozen, so that only the upper layers and the head were trained. This test was conducted on early fusion with 0, 6, 9, and 12 frozen layers out of the 12 RoBERTa layers, and 0, 3, and 6 out of the 6 DistilBERT layers. Partially frozen models were trained with three seeds

TABLE II
TRAINING SETTINGS

Setting	Value
Maximum input length	256 tokens
Batch size	32 (evaluation: 128)
Epochs	max. 3, best by validation macro-F1
Loss	cross-entropy
Optimizer	AdamW, weight decay 0.01
Learning rate schedule	10% linear warm-up, then linear decay
Gradient clipping	1.0
Classifier dropout	0.1 (RoBERTa), 0.2 (DistilBERT)
Precision	BF16 mixed precision
Learning rate grid (S1)	$\{1, 2, 3, 5\} \times 10^{-5}$
Seeds	42 (S1); 42, 43, 44 (final runs)
Selected learning rate (RoBERTa / DistilBERT)	
Comment only	1×10^{-5} / 3×10^{-5}
Early fusion	2×10^{-5} / 1×10^{-5}
Late fusion	1×10^{-5} / 1×10^{-5}

using the best learning rate from full fine-tuning. For fully frozen models, learning rates of 10^{-4} and 10^{-3} were tried with seed 42, and the better one was selected on validation data. Note that freezing reduces only the training cost, because all layers are still executed during inference.

D. Experimental Setup

The training settings are summarized in Table II. Biases and LayerNorm parameters were excluded from weight decay, and each batch was padded only to the length of its longest sequence. The epoch with the highest validation macro-F1 was used. The learning rate was determined through a grid search because RoBERTa is known to be sensitive to training settings [4]. The grid includes the values 2×10^{-5} and 3×10^{-5} used in [11], [10].

In total, 60 runs were carried out in three scenarios: a learning rate search with seed 42 (S1, 24 runs), layer freezing (S2, 16 runs), and repetition with seeds 43 and 44 (S3, 20 runs). Of these, 29 runs were evaluated on the test data, consisting of 6 S1 runs, 11 S2 runs, and 12 S3 runs. The remaining runs either used a learning rate that lost on validation data or still used a learning rate from early trials before the learning rate search was completed (11 runs). All runs were executed on a single NVIDIA GeForce RTX 5070 Ti 16 GB GPU. Training time was computed per epoch excluding validation, and the recorded memory is the peak GPU memory during training. Latency was measured as the forward pass time with inputs already on the GPU. The measurement used 1,000 validation examples at batch sizes 1 and 32, with the first ten batches treated as an uncounted warm-up, and the results are reported per example. Efficiency figures were averaged over the same final runs that were evaluated on the test data.

E. Evaluation and Statistical Testing

A comment is predicted as sarcastic if its probability is 0.5 or higher. The reported metrics are accuracy, precision, recall, and F1 for the sarcastic class, plus macro-F1 as the main metric. Epoch and learning rate selection used only validation data. Test predictions were stored without being scored, and the list of 29 final runs was recorded in a manifest file committed to the project repository on 16 October 2026, before any test score was computed. Scores are reported as the mean and standard deviation over seeds 42, 43, and 44, except for fully frozen models, which have only one run.

All comparisons were tested with one-sided hypotheses [13]. For example, in the parent comment comparison, H_0 states that macro-F1 with the parent is not higher than without it, whereas H_1 states the opposite. The tested data are the per-seed test macro-F1 scores, paired by seed and model. Comparisons of two configurations were tested with a one-sided paired t-test. The one-sided Wilcoxon signed-rank test was added only for pooled comparisons, because with three pairs the smallest p-value Wilcoxon can reach is 0.125. Comparisons of three or more treatments, namely the three inputs, the three seeds, and the four learning rates, were tested with the Friedman test. Effect size is reported as the mean difference with a two-sided 95% confidence interval (95% CI) for two-configuration tests, and as Kendall's W for the Friedman test. Normality of the differences in pooled comparisons was checked with the Shapiro-Wilk test. Given the number of tests, p-values were corrected with the Holm method within each question group, at a significance level of 0.05. Computational cost was tested with the same procedure. As a supporting test, run pairs were also compared at the level of individual test examples using McNemar's test with Holm correction. In addition, an exploratory analysis compared early fusion with comment only on several test subgroups, namely by comment length, comments that appear with different labels, and the confidence of the RoBERTa comment-only model. The model is considered uncertain when its mean probability lies in the range 0.4–0.6.

III. RESULTS

A. Performance and Efficiency of Both Models

The test scores and computational costs of the six main configurations are presented in Table III. The highest values on all metrics were obtained by RoBERTa with early fusion, with a macro-F1 of 79.62%. Across the three inputs, RoBERTa outperformed DistilBERT by 1.65–2.07 points. This advantage appeared in all nine seed and input pairs. The pooled difference was 1.81 points (95% CI 1.65–1.97) and remained significant after Holm correction (Table IV). The results were also stable. The standard deviation of accuracy and macro-F1 did not exceed 0.09 points, the macro-F1 difference between validation and test data ranged only from -0.07 to 0.26 points, and the Friedman test found no difference across seeds ($p = 0.311$). Unlike the seed, the learning rate had a significant effect ($p_{\mathrm{Holm}} = 0.004$). A learning rate of 5×10^{-5} always produced the lowest validation macro-F1, with the largest drop for RoBERTa.

In terms of efficiency, RoBERTa has 1.9 times more parameters, and its computational cost is roughly twice that of DistilBERT for all inputs. RoBERTa training took 1.92–1.94 times longer, its memory requirement was 1.93–1.96 times larger, and its batch-1 latency was 1.99–2.09 times higher. This pattern appeared consistently in all nine seed and input pairs

TABLE III
TEST SCORES AND COMPUTATIONAL COSTS OF THE MAIN CONFIGURATIONS

Input	Model	Test score (%) Accuracy	Precision	Recall	F1	Macro-F1	Training (min/epoch)	Latency (ms) bs 1	bs 32	Memory (GB)
Comment only	DistilBERT	76.88 ± 0.08	76.94 ± 0.65	75.80 ± 1.17	76.36 ± 0.29	76.87 ± 0.08	8.56	4.19	0.15	2.63
	RoBERTa	78.53 ± 0.07	78.33 ± 0.36	77.99 ± 0.46	78.16 ± 0.05	78.52 ± 0.06	16.61	8.42	0.31	5.09
Early fusion	DistilBERT	77.58 ± 0.06	78.14 ± 1.11	75.70 ± 2.03	76.88 ± 0.53	77.55 ± 0.09	9.93	4.23	0.32	2.64
	RoBERTa	**79.63** ± 0.05	**79.37** ± 1.31	**79.32** ± 2.22	**79.32** ± 0.48	**79.62** ± 0.06	19.03	8.41	0.61	5.11
Late fusion	DistilBERT	77.60 ± 0.02	77.62 ± 0.07	76.61 ± 0.07	77.12 ± 0.02	77.59 ± 0.02	14.92	7.20	0.42	4.29
	RoBERTa	79.30 ± 0.02	78.82 ± 0.21	79.29 ± 0.39	79.06 ± 0.09	79.30 ± 0.02	28.82	15.06	0.83	8.38

Early and late fusion use comment + parent. Scores are mean ± standard deviation over seeds 42, 43, and 44, and costs are averaged over the same runs. Precision, recall, and F1 are computed for the sarcastic class. Latency at batch size (bs) 32 is reported per example.

TABLE IV
HYPOTHESIS TEST RESULTS ON TEST DATA

H_1	n	Δ	95% CI	p_{Holm}	p_W	McN.
Performance: RoBERTa > DistilBERT						
Pooled, 3 inputs	9	1.81	1.65–1.97	<0.001	0.002	yes
Layer freezing: full fine-tuning > partially frozen						
RoBERTa, 6 layers	3	0.24	−0.02–0.49	0.028	–	no
RoBERTa, 9 layers	3	1.00	0.81–1.18	0.002	–	–
DistilBERT, 3 layers	3	1.47	1.28–1.66	0.002	–	–
Pooled	9	0.90	0.48–1.32	0.002	0.002	–
Input: difference among the three inputs (Friedman)						
Comment, early, late	6	$W = 0.78$		0.009	–	–
Parent comment: comment + parent > comment only						
Early, RoBERTa	3	1.10	0.83–1.37	0.004	–	yes
Early, DistilBERT	3	0.69	0.65–0.72	<0.001	–	yes
Late, RoBERTa	3	0.78	0.60–0.96	0.004	–	yes
Late, DistilBERT	3	0.72	0.53–0.92	0.004	–	yes
Early, pooled	6	0.89	0.64–1.14	<0.001	0.016	–
Late, pooled	6	0.75	0.67–0.83	<0.001	0.016	–
Fusion strategy: early fusion > late fusion						
RoBERTa	3	0.32	0.15–0.49	0.023	–	yes
DistilBERT	3	−0.04	−0.26–0.19	0.725	–	no
Pooled	6	0.14	−0.08–0.36	0.158	0.109	–
Cost: RoBERTa > DistilBERT (pooled, 3 inputs)						
Training (min/epoch)	9	10.35	8.27–12.43	<0.001	0.002	–
Latency bs 1 (ms)	9	5.42	4.01–6.82	<0.001	0.002	–
Memory (GB)	9	3.01	2.38–3.64	<0.001	0.002	–
Supporting (Friedman)						
Three seeds	6	$W = 0.19$		0.311	–	–
Four LRs (validation)	6	$W = 0.74$		0.004	–	–

n: number of pairs or blocks; Δ: mean difference (macro-F1 points, or cost units); 95% CI: two-sided, so it may slightly include zero even when the one-sided test is significant; p_{Holm}: paired t-test or Friedman p after Holm correction within each question group; p_W: Wilcoxon; McN.: significant under McNemar with Holm correction for all three seeds (–: not tested).

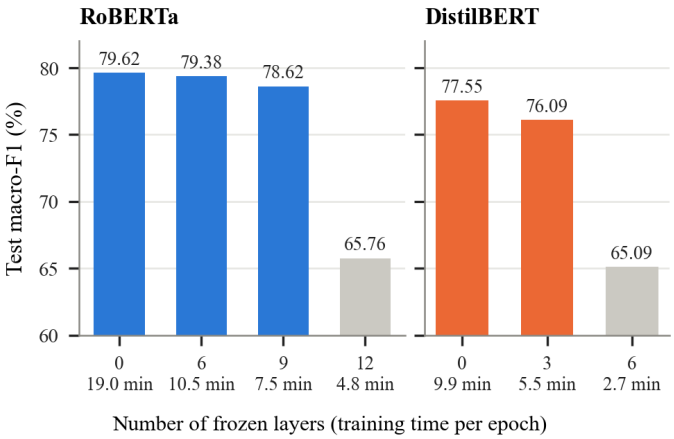


Fig. 1. Test macro-F1 by number of frozen layers in early fusion. The numbers below the axis show the training time per epoch. Grey bars are fully frozen models (one seed); the other bars are means over three seeds.

(Wilcoxon, $p = 0.002$ for all three measures) and remained significant after Holm correction.

B. Effect of the Number of Frozen Layers

Fig. 1 shows macro-F1 and training time for each number of frozen layers. When 6 of the 12 RoBERTa layers were frozen, macro-F1 dropped only from 79.62% to 79.38%. This 0.24-point drop occurred in all three seeds and was significant under the one-sided paired t-test ($p_{\text{Holm}} = 0.028$), with a one-sided 95% CI lower bound of 0.07 points. However, the two-sided 95% CI (−0.02–0.49) still slightly crosses zero and McNemar did not find it significant, so the drop can be considered small. On the cost side, training time fell from 19.03 to 10.51 min per epoch, and memory fell from 5.11 to 2.70 GB. With 9 frozen layers, RoBERTa still reached 78.62%. This is 1.07 points higher than fully fine-tuned DistilBERT, even though its training time was shorter (7.49 vs. 9.93 min) and its memory was smaller (1.67 vs. 2.64 GB). DistilBERT turned out to be more sensitive to freezing, as freezing only 3 of its 6 layers already reduced macro-F1 by 1.47 points. When all layers were frozen, macro-F1 of both models fell to around 65%. Latency remained relatively unchanged, at 7.80–8.41 ms for RoBERTa and 3.91–4.23 ms for DistilBERT, and the latency reduction due to freezing was not significant after Holm correction ($p_{\text{Holm}} = 0.079$).

C. Effect of the Parent Comment

The Friedman test showed a difference in macro-F1 among the three inputs ($p = 0.009$, Kendall's $W = 0.78$). As shown in Fig. 2, adding the parent increased macro-F1 for both models. With early fusion, the gain was 1.10 points for RoBERTa and 0.69 points for DistilBERT, while late fusion gave gains of 0.78 and 0.72 points. All four gains were significant, both under the paired t-test after Holm correction and under McNemar. The additional cost of early fusion was small. Training time increased by only 1.15–1.16 times, while batch-1 latency and memory barely changed. Only the per-example latency at batch size 32 increased to roughly twice.

The benefit of the parent was not uniform across comments. On the 10,737 comments for which the RoBERTa comment-

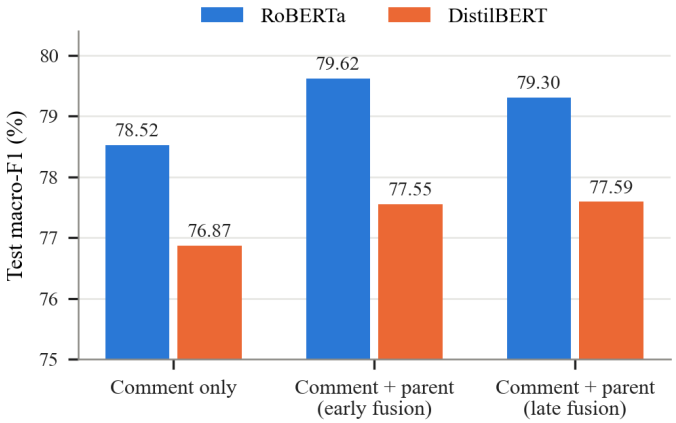


Fig. 2. Test macro-F1 for each model and input configuration (mean over three seeds). The vertical axis starts at 75% so that differences between configurations are visible.

only model was uncertain, early fusion increased RoBERTa macro-F1 by 8.18 points, whereas on confidently predicted comments the gain was only 0.25 points. The gain on short comments of 1–3 words (2.59 points) was also larger than on comments longer than 10 words (0.71 points). On the 5,886 comments (593 unique sentences) that appear with different labels, the gain reached 2.73 points. Even so, the parent did not always help. For RoBERTa, the parent corrected an average of 6,236 predictions per seed but also turned 5,119 other predictions wrong.

D. Best Fusion Strategy

For RoBERTa, early fusion produced the highest macro-F1 and outperformed late fusion by 0.32 points (95% CI 0.15–0.49, $p_{\text{Holm}} = 0.023$). This result was also significant under McNemar. For DistilBERT, the two strategies gave equivalent results (77.55% and 77.59%). When both models were pooled, the advantage of early fusion was not significant ($p_{\text{Holm}} = 0.158$, 95% CI -0.08–0.36). In terms of cost, late fusion was the most expensive configuration, with about 1.5 times the training time, 1.62–1.64 times the memory, and 1.70–1.79 times the batch-1 latency of early fusion.

IV. Discussion

A. Trade-off between RoBERTa and DistilBERT

The test results provide statistical evidence that RoBERTa is more accurate. However, its advantage is only 1.65–2.07 points, while its cost is about twice that of DistilBERT. In other words, DistilBERT retains 97–98% of the RoBERTa macro-F1 with roughly half the resources. This cost ratio is nearly the same for all three inputs, so it is most likely determined by model size. RoBERTa is more suitable when accuracy is the priority and resources are available, for example in offline analysis. Conversely, DistilBERT is more suitable when latency and memory are limited, such as in moderation tools that must process comments continuously.

B. Layer Freezing and Distillation

The test results support an effect of the number of frozen layers on performance, although the effect is not linear. Freezing half of the RoBERTa layers reduced macro-F1 by only 0.24 points, while training time fell by 45% and memory by 47%. A clear drop appeared only at 9 layers and became very large when all layers were frozen, so the pretrained representations still need to be fine-tuned. DistilBERT performance dropped faster, probably because it has fewer layers. In addition, freezing and distillation save different parts of the cost. Freezing saves only training cost and therefore suits models that need frequent retraining, whereas DistilBERT saves inference cost because its latency is about half that of RoBERTa.

C. Role of the Parent Comment

The test results provide statistical evidence that the parent comment improves the performance of both models, although the average gain is only about one point. This small gain is reasonable, since most comments can already be understood from their own content. The parent matters mainly when a comment is ambiguous. An example is the reply ";)", which appears 68 times in the test data, of which only six are sarcastic. As a reply to "Sarcasm doesn't translate well through text. Haha", this comment is labeled sarcastic, whereas as a reply to "What is your Guilty Pleasure?", the same comment is not sarcastic. Without the parent, the model has no choice but to give the same prediction for both cases. The gain obtained here is much smaller than that reported by Dubey et al. (7.1 points [11] and 6.8 points [7]), who used a subset of 7,370 samples with punctuation and capitalization removed. A comment-only model without punctuation may rely more on the parent, but this hypothesis was not tested in this study.

D. Early Fusion and Late Fusion

Early fusion was chosen as the best strategy because there is statistical evidence that it is more accurate for RoBERTa, it is not significantly different for DistilBERT, and its cost is much lower. In early fusion, comment tokens can attend to specific words in the parent at every layer. In late fusion, by contrast, both texts are first compressed into a single vector each and only meet at the classification head. Since the meaning of sarcasm is often determined by specific phrases in the parent, late fusion appears to be less advantageous. Late fusion may still be useful when one parent receives many replies, because the parent vector needs to be computed only once, but this scenario was not tested.

E. Comparison with Other Studies and Limitations

Several studies have reported higher scores, for example an F1 of 95.6% in [11] and 0.917 in [12]. However, the data size, preprocessing, and evaluation protocol in those studies differ. The "/s" fragments described in Section II-A may also play a role, because almost all of them are labeled sarcastic, while none of the cited reports state whether such fragments were removed. The cost ratio, on the other hand, is consistent: the RoBERTa training time on SARC in [11] is 1.76 times that

of DistilBERT (8.1 and 4.6 hours), compared with 1.92–1.94 times in this study.

The answers to the three research questions apply only to the dataset, settings, and three seeds used, and this study has several limitations. First, training was limited to three epochs, even though 24 of the 29 final runs reached their best validation score at the last epoch. Second, only one dataset was used, and its labels come from the "/s" marker, so sarcastic comments without the marker are recorded as non-sarcastic [5]. The official SARC test split also could not be used because its text is not available. Third, the frozen models used the learning rate from full fine-tuning, the fully frozen models were run with only one seed, and each per-configuration test had only three pairs. Finally, large language models were not included, and all efficiency measurements were performed on a single GPU.

V. Conclusion

This study examined the trade-off between performance and computational efficiency of RoBERTa and DistilBERT for sarcasm detection on SARC. The test results provide statistical evidence that RoBERTa is more accurate, but its advantage is only about two macro-F1 points, while its computational cost is roughly twice that of DistilBERT. The number of frozen layers affects performance, but half of the RoBERTa layers can still be frozen with a very small drop to save training cost. Adding the parent comment improves the performance of both models, and early fusion is the best strategy because it is the most accurate for RoBERTa and cheaper than late fusion. Therefore, RoBERTa with early fusion is recommended when accuracy is the priority, whereas DistilBERT with early fusion is recommended when latency and memory are limited. Future work could extend training, search for learning rates specific to frozen models, test other datasets, and compare the results with large language models.

References

[1] O. Prakash, "Sarcasm detection for sentiment analysis using deep learning enhancement," *Procedia Computer Science*, vol. 258, pp. 203–212, 2025.

[2] R. Dhumpati, A. Sasi, S. J. Basha, S. Vadhri, P. Saroja, S. Alluri, and R. Vatambeti, "Enhancing sarcasm detection in sentiment analysis for cyberspace safety using advanced deep learning techniques," *Scientific Reports*, vol. 15, p. 22681, 2025.

[3] V. Grover and H. Banati, "An emoji centric approach to sarcasm detection in online discourse," *Scientific Reports*, vol. 16, p. 3891, 2026.

[4] R. Jiménez Castro, J. P. Sánchez-Solís, V. García Jiménez, G. Rivera Zárate, and R. Florencia Juárez, "A comparative study of bert-based models for sarcasm detection in social media texts," *International Journal of Combinatorial Optimization Problems and Informatics*, vol. 17, no. 2, pp. 378–390, 2026.

[5] M. Khodak, N. Saunshi, and K. Vodrahalli, "A large self-annotated corpus for sarcasm," in *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*. Miyazaki, Japan: European Language Resources Association (ELRA), 2018.

[6] T. Shangipour Ataei, S. Javdan, and B. Minaei-Bidgoli, "Applying transformers and aspect-based sentiment analysis approaches on sarcasm detection," in *Proceedings of the Second Workshop on Figurative Language Processing*, 2020, pp. 67–71.

[7] P. Dubey, P. Dubey, and P. N. Bokoro, "Unpacking sarcasm: A contextual and transformer-based approach for improved detection," *Computers*, vol. 14, no. 3, p. 95, 2025.

[8] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[9] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[10] M. A. Ullah, R. Sultana, S. H. Chowdhury, and F. A. Laila, "Advanced sarcasm detection using transformer-based models: Analyzing variations in formal and informal languages," *Applied Computational Intelligence and Soft Computing*, vol. 2026, p. 8761185, 2026.

[11] P. Dubey, A. Mishra, A. Singh, Murtuza, A. Chanchlani, and P. Dubey, "Efficient transformer architecture for sarcasm detection: A study on compression and performance," *Bulletin of Electrical Engineering and Informatics*, vol. 15, no. 1, pp. 557–567, 2026.

[12] L. Zhang, M. Faseeh, S. S. A. Naqvi, L. Hu, and A. Ghani, "Enhancing sarcasm detection on social media: A comprehensive study using llms and bert with multi-headed attention on sarc," *PLoS ONE*, vol. 20, no. 11, p. e0334120, 2025.

[13] R. Dror, L. Peled-Cohen, S. Shlomov, and R. Reichart, *Statistical Significance Testing for Natural Language Processing*, ser. Synthesis Lectures on Human Language Technologies. Cham, Switzerland: Springer, 2020.